Pytorch Neural Networks

```
device = torch.accelerator.current_accelerator().type if torch.accelerator.is_available() else "cpu"
print(f"Using {device} device")
```

```
ut:
```

Basically runs on the GPU accelerator for mac fcause it

Define the Class

We define our neural network by subclassing `nn.Module`, and initialize the neural network layers in `__init__`. Every `nn.Module` subclass implements the operations on input data in the `forward` method.

```
class NeuralNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()
        self.linear_relu_stack = nn.Sequential(
            nn.Linear(28*28, 512),
            nn.ReLU(),
            nn.Linear(512, 512),
            nn.ReLU(),
            nn.Linear(512, 10),
        )

    def forward(self, x):
        x = self.flatten(x)
        logits = self.linear_relu_stack(x)
        return logits
```

making own NN from a library of NN nn.module is one of them

originally grid this turns into Linear Layer

How it should travel through NN

X: Fashion MNST Image

Put x through function

Logit: 10 numbers basically predicts how much a model favors a class [-0.3, 0.5, 4.8]

Bag

Builds the NN 784 in put to 512 new numbers
↳ organizing to find patterns

Adds non-linearity which prevents model overfitting

Model can predict Bag with good accuracy

Then compare Logits with correct answer

Calling the model on the input returns a 2-dimensional tensor with dim=0 corresponding to each output of 10 raw predicted values for each class, and dim=1 corresponding to the individual values of each output. We get the prediction probabilities by passing it through an instance of the `nn.Softmax` module.

```
X = torch.rand(1, 28, 28, device=device)
logits = model(X)
pred_probab = nn.Softmax(dim=1)(logits)
y_pred = pred_probab.argmax(1)
print(f"Predicted class: {y_pred}")
```

Out:

```
Predicted class: tensor([3], device='cuda:0')
```

Dim: 0 = Image
Dim: 1 = classes
dress, sock, t-shirt

Turns Logits to probabilities

Tells position of biggest number

nn.Linear

The linear layer is a module that applies a linear transformation on the input using its stored weights and biases.

```
layer1 = nn.Linear(in_features=28*28, out_features=20)
hidden1 = layer1(flat_image)
print(hidden1.size())
```

Already discussed but takes certain input numbers and turn into lesser output numbers

The logic is making neurons, turns 784 inputs into 20 outputs
Neurons who have 784 weights

Model Parameters

Many layers inside a neural network are *parameterized*, i.e. have associated weights and biases that are optimized during training. Subclassing `nn.Module` automatically tracks all fields defined inside your model object, and makes all parameters accessible using your model's `parameters()` or `named_parameters()` methods.

In this example, we iterate over each parameter, and print its size and a preview of its values.

```
print(f"Model structure: {model}\n\n")

for name, param in model.named_parameters():
    print(f"Layer: {name} | Size: {param.size()} | Values : {param[:2]} \n")
```

Out:

Basically a neuron has two inputs

$$output = (x_1 \cdot w_1) + (x_2 \cdot w_2) + b$$

bias, weight are things that the model will change if it gets a bad prediction.

Each neuron has 784 weights + Bias that change